

从 GCN 到 EvolveGCN

代码增量审计 + 论文段落—代码对应分析

研究对象：GCN → GCRN → EvolveGCN，并以 DynamicGEM 系列作横向比较

作者：Junxiang Zeng 日期：2026-07-15

报告定位

完整保留阶段一，并追加阶段二的逐段文本功能分析

摘要

这份报告不从论文的公式包装出发，而从可执行代码追问一个更具体的问题：**GCN、GCRN 与 EvolveGCN 之间，跨时间保存的状态究竟是什么？**对四个仓库、三套框架进行语义级定位后，核心变化可以压缩成一条清楚的接线演化：静态 GCN 固定学习一组卷积权重；GCRN 在节点表示外面套循环网络；EvolveGCN 则让循环网络直接演化 GCN 权重。后一个改动把跨时刻状态从依赖节点数的矩阵，换成只依赖特征维度的矩阵，因此能够自然面对节点出现与消失。

一句话结论 EvolveGCN 的代码创新不在于“又加了一个 RNN”，而在于把 RNN 接到了 **GCN 权重** 上。核心实现只有几十行，但它改变了模型对动态图节点集合的依赖方式。

审计范围与证据口径

本报告锁定以下代码来源。行号按分析时的仓库快照记录；不同仓库使用 TensorFlow、Keras 与 PyTorch，不能依赖文件名直接 diff，因此审计以“变量在计算图中的角色”作为共同语言。

角色	仓库/实现	框架	主要证据
静态基线	tkipf/gcn	TensorFlow	layers.py 中图卷积权重与传播式
循环基线	GCRN / GCN-GRU	PyTorch 等	逐时刻 GCN 表示被堆叠后送入 LSTM/GRU
研究对象	IBM/EvolveGCN	PyTorch	egcn_h.py 、 egc_n_o.py 、 mat_GRU_gate.py
横向比较	DynamicGEM	Keras	邻接矩阵自编码重构与动态图嵌入

证据分成三层：第一层是模型主前向路径，决定“数据究竟怎么流”；第二层是适配模块，如 TopK 与矩阵 GRU，解释可变节点集合如何被固定尺寸运算接住；第三层是训练目标与输出任务，用来区分技术祖先与仅被比较的方法。报告结论优先采用第一层证据。

四步看清算法演化

第一步：原始 GCN——权重训练后固定

在 tkipf/gcn 的 layers.py 中，每一层创建权重矩阵 weights_i。省略偏置、dropout 与激活后，前向计算可以写成：

$$H^{l+1} = \sigma(\tilde{A}H^lW^l)$$

其中 H 是节点特征， W 是训练得到的层权重， A 是当前图的归一化邻接矩阵。一次训练结束后， W 就作为固定参数使用。这个模型可以处理一张图上的节点聚合，却没有任何跨时间状态，也不知道“上一时刻”是什么。

跨时间保存的东西 没有。模型参数 W 在一次前向传播中不随时间步变化，时间也没有进入模型定义。

第二步：GCRN / GCN-GRU——在节点表示外套循环

常见动态图基线先在每个时刻执行一次 GCN，得到节点嵌入 z_t ，再用 torch.stack 把各时刻结果拼成序列并交给 LSTM/GRU。抽象后是：

$$Z_t = \text{GCN}(A_t, X_t; W), \quad S_t = \text{RNN}(Z_t, S_{t-1})$$

第三方 GCRN 实现中的循环状态形状带有 `nNode` 维度，即每个节点有自己的隐藏状态。它表达了“同一个节点随时间如何变化”，但也把跨时间记忆与节点集合绑定在一起：如果节点数或节点身份改变，旧状态就不再能直接对齐新图。

这里需要保留一个证据边界：用于代码形态说明的第三方 GCRN 仓库晚于原论文，适合作为“典型 GCN-RNN 接法”的实现证据，不应冒充原论文的官方逐行实现。因此这一段的实现对应置信度标为中等。

第三步：EvolveGCN——让循环网络演化权重

`egcn_h.py` 的关键路径可以压缩成两步：

```
GCN_weights = evolve_weights(previous_weights, node_embeddings)
node_embs = activation(Ahat @ (node_embs @ GCN_weights))
```

第一步用矩阵 RNN 根据上一时刻的 GCN 权重以及当前节点信息产生新权重；第二步再用新权重执行图卷积。于是跨时刻保存的状态不再是每个节点的 $N \times d$ 表示，而是层权重 $F_{in} \times F_{out}$ ：

模型	跨时刻状态	尺寸依赖	节点增减
GCN	无	—	没有时间建模
GCRN	节点隐藏状态	$N \times d$	需要节点对齐
EvolveGCN	GCN 层权重	$F_{in} \times F_{out}$	当前步可用任意 N_t

节点数 N_t 仍然出现在当前时刻的邻接矩阵与节点特征中，但不再进入跨时刻状态的形状。这就是 EvolveGCN 能处理节点出现、消失与集合变化的直接代码原因。

H 版：TopK 是尺寸适配器

EvolveGCN-H 更新权重时需要读取当前节点嵌入，但节点数会变，而矩阵 GRU 的输入尺寸固定。TopK 模块先对节点打分，选出固定数量的代表节点，并把不足部分补齐，最终形成固定宽度矩阵。它不是一条旁支技巧，而是可变 N_t 与固定权重尺寸之间的接口。

TopK 的真正作用 它不是为了普通意义上的池化精度，而是把“数量会变的节点集合”整理成矩阵 GRU 能接收的固定形状输入。

O 版：只由旧权重生成新权重

EvolveGCN-O 更激进：更新 `GCN_weights` 时不读取当前节点嵌入，只让权重沿时间自主演化。H 版可以理解为“权重状态 + 当前图信号”，O 版则是“权重状态自身的时间模型”。两者共享“演化权重而非节点状态”的总思想，但信息入口不同。

一个值得记录的论文—代码差异

论文叙述常用 LSTM 类比 O 版的权重演化；官方代码实际沿用 `mat_GRU_cell` 与矩阵门控实现。概念层面仍是门控循环演化权重，但具体单元不是字面上的标准 LSTM。这个差异不推翻论文主张，却说明只读论文容易把实现细节想得过于整齐。

第四步：DynamicGEM——同场比较，不是接线祖先

DynamicGEM 的 `sdne_dynamic.py`、`dynAE.py` 等实现主要用自编码器重构邻接结构。典型输入维度直接与 `node_num` 相关，优化目标是重构误差；EvolveGCN 则执行图消息传递，并围绕节点分类/链路预测等任务训练。两者都处理动态图，但方法论路线不同：

路线	核心运算	主要目标	谱系判断
EvolveGCN	动态图上的图卷积 + 权重演化	任务损失	GCN/GCRN 接线演化
DynamicGEM	邻接/结构自编码重构	重构与嵌入损失	横向比较方法

因此本报告把 DynamicGEM 标成 `COMPARES_WITH`，而不是 EvolveGCN 的技术祖先。标题、应用领域或引用关系相近，不等于主计算路径相承。

从代码行到研究判断

核心创新到底有多大

从实现体积看，EvolveGCN 相对典型 GCRN 的核心变化确实只有几十行：把循环单元的输入输出对象由节点嵌入换成 GCN 权重，再增加一层尺寸适配。这个判断承认“代码增量小”，但不等于“研究价值为零”。真正有价值的是那个接线选择带来的不变量：跨时间状态不再依赖 N 。

论文工程量也不能只按核心 diff 计算。为证明这几十行有效，作者仍需完成动态图数据处理、负采样或任务构造、基线对齐、调参、消融、重复实验、图表与论文写作。这些工作支撑可信度，却不是算法定义本身。

这套审计流水线做了什么

- 锁定论文与仓库快照，避免把同名或后来的实现混在一起；
- 为不同框架建立语义统一的代码地图，定位主前向路径、状态形状和损失；
- 把代码实体组织成 `EXTENDS`、`REPLACES`、`COMPARES_WITH` 等关系；
- 对每个核心判断保留文件、行号、变量名和置信度；
- 将“论文声称什么”与“代码实际执行什么”分开记录。

这使得审计结果不是一句“大模型觉得它们相似”，而是一条可以回到源码复核的证据链。

置信度与限制

静态 GCN 与 EvolveGCN 的核心结论来自官方仓库的主路径，置信度高；GCRN 的具体 state 形状来自典型第三方实现，置信度中等；DynamicGEM 的自编码路线来自官方项目代码与论文描述，足以支持“比较而非祖先”的判断。行号会随仓库更新漂移，公开结果同时保留结构化代码地图，方便用变量名和上下文再次定位。

阶段一结论 真正需要从一篇长论文中提取的，不只是“提出了一个新模型”，而是：它改动了哪一个状态、哪一条数据流、哪一个尺寸依赖，以及这些变化解决了什么约束。EvolveGCN 的答案是：演化权重，从而把跨时间状态与节点数量解耦。

可复核材料

项目仓库保存两类结果：`results/stage1/technical_increment_report.md` 给出完整技术增量报告，`results/stage1/repo_maps/` 保存四个仓库的结构化语义地图。复现脚本位于 `code/`。出于版权与安全边界，仓库不重新分发论文全文、原始 PDF、私有 API 配置或本机路径。

本版本回答的是“代码到底改了什么”。下一阶段在不改变本报告任何阶段一正文的前提下，继续回答“这些代码创新在论文中占了多少段，每段承担什么叙事功能”。

阶段二：代码创新在论文里占了多少文字

版本关系 本章是 V2 新增内容。前面的阶段一正文直接包含自 V1 的同一个源文件，没有删节或改写；V2 只在其后追加“段落—代码对应”分析。

分析问题与分类方法

阶段二把五篇论文的正文切成段落，逐段回答两个问题：第一，这一段在论文叙事中想完成什么任务；第二，它与可执行代码的距离有多近。分析对象包括 GCN、GCRN、EvolveGCN、DynamicGEM 与 dyngraph2vec，共 274 个正文段落、23,151 个英文词。

每段首先按功能标注为问题背景、相关工作、方法动机、算法定义、训练目标、实验设置、结果解释、局限或结论等；随后归入四级代码关系：

关系	含义	典型内容
DIRECT	直接对应	前向传播、状态更新、损失函数、明确的实现步骤
SUPPORT	实现支撑	数据预处理、训练流程、复杂度、任务接口与超参数
RATIONALE	设计理由	为什么要这样接线、旧方法的限制、理论或直觉解释
NO_CODE	无直接对应	领域背景、文献综述、结果讨论、写作性总结

“核心创新段落”是更严格的子集：只有直接定义或解释论文独有算法增量的段落才计入。实验段落即使非常重要，也不会因为提到模型名就被算作核心代码。

总体结果：核心代码只占约五分之一



五篇论文中，核心创新共 52 段、4,197 词，占全部词数的 18.1%。换句话说，读者看到的长论文里，大约五分之一直接承担“核心算法是什么”的工作，其余文字负责把问题立住、把选择讲通、把实验做可信、把结果放进学术语境。

论文	段落	词数	核心段	核心词占比
GCN	62	4,984	10	17.1%
GCRN	30	3,628	7	13.5%
EvolveGCN	57	4,319	15	21.8%
DynamicGEM	56	4,595	10	22.1%
dyngraph2vec	69	5,625	10	16.0%
合计	274	23,151	52	18.1%

四类文字分别占多少

代码关系	段落	词数占比	它在论文中的任务
DIRECT	56	20.5%	把模型写成公式、伪代码、状态更新或损失
SUPPORT	67	17.9%	说明训练、数据、复杂度与实现条件
RATIONALE	49	22.4%	解释为什么这项改动有必要、合理
NO_CODE	102	39.2%	建立背景、比较文献、解释结果与总结贡献

这张表揭示了一个常被忽略的事实：与代码没有直接对应并不等于废话。RATIONALE 与 NO_CODE 合计超过六成，承担的是“为什么值得做”和“结果意味着什么”。如果只看代码 diff，会迅速抓住机制；如果要判断证据是否可靠，仍必须回到这些段落。

每篇论文的核心段落到底在讲什么

GCN：把谱卷积压缩成局部传播规则

GCN 的核心段落主要完成三件事：从谱图卷积出发建立理论入口；通过一阶近似把全局谱运算简化；给出归一化邻接矩阵上的层传播规则。代码里最终留下的只是 $A \times (X \times W)$ 一类稀疏矩阵运算，但论文需要解释这个简单式子为什么不是随意拼出来的，以及归一化如何避免数值与尺度问题。

非核心段落则介绍半监督节点分类、引用网络数据集、训练设置、与已有方法的精度比较。它们不改变前向代码，却负责证明这个简化公式在真实任务上有效。

GCRN：说明“图结构 + 时间循环”如何组合

GCRN 的核心文字比例最低，为 13.5%。关键段落把图卷积嵌入循环单元，说明输入到状态如何同时吸收空间邻接与时间依赖。对应代码通常体现为逐时刻图卷积、序列堆叠和带节点维度的隐藏状态。

其余篇幅大量用于问题定义、已有时空模型、数据场景与预测实验。原因很直接：把两个成熟模块接在一起的代码不长，但论文必须说明为什么这种组合比纯 GCN 或纯 RNN 更适合特定时空任务。

EvolveGCN：围绕“演化参数而非节点状态”展开

EvolveGCN 的 15 个核心段落集中讲四件事：动态图节点集合为什么使节点级循环状态受限；GCN 参数如何被视为时间状态；H/O 两个变体如何更新权重；H 版怎样通过 TopK 把可变节点集合变成固定尺寸输入。这些文字与阶段一定位到的 `evolve_weights`、矩阵 GRU 和 TopK 直接对应。

没有直接代码对应的段落则在介绍动态图任务、组织相关工作、报告链路预测与节点分类结果、分析不同数据集上 H/O 的表现。论文的“长”，主要来自为一个简洁 insight 建立问题必要性和经验证据，而不是隐藏了大量额外算法。

DynamicGEM：搭建方法集合与统一评测框架

DynamicGEM 的核心段落介绍动态图嵌入问题、稳定性目标以及若干增量更新路线；代码支撑段落还要处理快照、节点映射、评估接口和多种算法封装。它更像一个方法库与基准框架，因此文章篇幅不只服务于单一新算子。

这也进一步支持阶段一的谱系判断：DynamicGEM 与 EvolveGCN 的相邻之处是问题领域和实验比较，而不是从 GCN-RNN 接线逐步演化而来。

dyngraph2vec：用深度自编码器建模多步历史

dyngraph2vec 的核心段落定义邻接历史窗口、全连接或循环自编码结构以及重构目标；其余文字解释动态图嵌入背景、数据集、链路预测评测和不同历史长度的影响。这里的关键代码对象是邻接向量和重构网络，而不是 EvolveGCN 的图卷积权重。

为什么几十行改动能写成十几页论文

论文篇幅可以拆成五项互补工作：

1. **问题合法性**：证明旧模型确实在某个条件下受限，例如节点集合变化让节点状态难以对齐。
2. **机制定义**：用公式、伪代码和少量核心实现准确写出新接线。这通常是最短的一部分。
3. **设计解释**：说明为什么选择演化权重、为什么需要 TopK、两个变体有什么差异。
4. **证据建设**：数据集、基线、公平设置、消融、敏感性和统计结果，回答“它真的有效吗”。
5. **学术定位**：与相关工作区分边界，承认限制，并说明结果能否推广。

对‘论文是不是只改了一点’的准确回答 是，许多论文的**算法核心**确实可以压缩成一个接线变化或几十行代码；但论文的其余部分是在为这个改动建立理由与证据。评价论文时应把“核心机制的原创性”与“实验和工程的完整性”分开打分，而不是混成一个模糊的“大贡献”。

对阅读与选题的实际价值

这套段落映射把阅读顺序改成了可检验流程：先从核心代码确认状态、尺寸和数据流；再只读 `DIRECT` 与核心创新段落，核对论文怎样定义它；随后读 `RATIONALE`，判断问题是否真实；最后读实验与 `NO_CODE` 段落，检查证据是否支撑主张。这样既不会被十几页叙事淹没，也不会因为代码短就错过重要的实验边界。

对于选题，可以重点追问：新工作改变的是哪一个不变量？现有仓库中最小可行 diff 是什么？这个 diff 解决的限制是否在数据上真实存在？需要哪些实验才能排除“只是换了接线”的替代解释？这些问题比单纯计算代码行数更接近研究判断。

数据质量与复现边界

段落切分、标题识别、词数统计和标签完整性经过自动校验；最终 274 个段落均有功能标签、代码关系、中文释义和来源位置。项目公开 `paragraph_inventory_public.csv`，保留段落编号、章节、词数、标签、中文概述与代码引用，不重新分发论文原文。完整统计与验证结果分别位于 `paper_statistics.json` 和 `paragraph_analysis_validation.json`。

需要注意，段落分类包含语义判断，不应被解读为绝对客观的计量学结论；不同排版版本也可能改变段落边界。18.1% 的意义是给出可复核的数量级，而不是宣称所有论文都存在固定比例。

最终结论 阶段一说明 EvolveGCN 到底改了什么：把时间状态从节点表示换成图卷积权重。阶段二说明长论文剩余的文字在做什么：约五分之一直接定义核心创新，其余部分主要建立动机、实现条件、实验可信度和学术位置。两部分合在一起，才能既看穿包装，又公平评价工作量。